Introduction

The primary plan for this project was mostly executed without any significant changes. Only minor changes are made, such as the implementation of the XWindow class to facilitate with the graphical display of the program, and the changes in the parameters of Card's apply() function (instead of taking in no parameters, it takes in Player objects, allowing card classes' access to Player's cards and their active status).

Most functions are executed within the specified dates in the plan, with tasks generally divided into design patterns and individual classes. Our group members initially worked independently on their respective classes, with frequent communication between group members. This approach helped reduce conflicts between members' implementations and ensured a smoother integration process. By maintaining regular updates and addressing issues promptly, we were able to achieve a cohesive final product.

Overview

In our Sorcery Card Game project, we focused on implementing several design patterns to structure and implement all the game features, particularly the Observer, and Decorator patterns.

The Board class serves as the central component of the game; it acts as the concrete subject in the Observer pattern. The Board manages the state of the game and notifies observers, including both a text-based observer and a graphical observer, of any changes. The Graphical Observer has an aggregate relationship with XWindow, to render visual elements through the X11 interface.

The Player class has a composite relationship with the Board; it includes and represents a single player in the game. Players have an aggregate relationship with Cards, which are represented by an abstract base class called Card. The Card class is the foundation for specific card types, including Ritual, Spell, and MinionComponent containing Minion and Enchantment, each inheriting from the Card class.

Minions are a key component of the game and inherit from the MinionComponent class. Some minions can have activated abilities, which are implemented with the help of ActivatedAbility class which has a composite relation with the Minion class.

In addition to that, minions can also have enchantments on top of it. This is implemented using a Decorator pattern acting as a linkedlist of Enchantments with Minion attached to the end. For this, the MinionComponent class utilizes the Decorator pattern to apply affects to the minions with the enchantments. In this pattern, the abstract base class MinionComponent acts as the abstraction component, the Minion as the concrete MinionComponent class, and the Enchantment as the concrete decorator. This design is used to efficiently add the enchantments, without changing any other functionality of the minions.

Overall, using the Observer and Decorator patterns has improved our Sorcery Card Game. These patterns ensure the game state is accurately triggered, players and cards are well-managed, and minions can be easily enhanced with new abilities.

Design: Decorator design pattern

One challenge in this project is the interaction between the Enchantment and Minion classes. The characteristic of the game is that multiple Enchantments can be added to a single Minion object, affecting or overshadowing the Minion's fields (e.g., attack, defense, abilities). This raises doubts about how the values in the objects should be accessed. The design that helps solve this problem is the Decorator pattern. We created an Abstract Base Class, the MinionComponent, acting as a parent class of Minion and another Abstract Base Class, the Decorator, which Enchantment inherits from. This maintains abstraction by ensuring that the other classes can only access an enchanted Minion's values through the MinionComponent. For example, retrieving the attack value of an enchanted Minion from outside the class triggers the pure virtual getAttack() function in MinionComponent; this redirects to the concrete getAttack() function in its list of Enchantments acting as the concrete Decorators, applying effects such as "+2" or "*2" each time. The Minion Class is always the last object at the end of the chain of Enchantments, returning the default fields in the Minion. This pattern ensures that the behavior of Minion is modified by the Enchantments without changing Minion's default values; it also guarantees Enchantment's oldest-to-newest application rule, since the effects of the oldest Enchantments (Enchantments that are closest to the Minion at the end) are applied first to the Minion's fields. Another example of this can be seen in the apply() function that calls the triggered abilities of the Minion. When an Enchantment such as "Silence" is applied to a Minion/enchanted minion, the minion's triggered ability should be disabled. To achieve this, as the apply() in MinionComponent is called, the top Enchantment in the decorator, "Silence", would first call its apply() function, successfully blocking the access to other Enchantments' or Minion's triggered abilities.

Design: Observer design pattern

In our project, another challenge was to display the board. Although it was fairly easy to display the board on the command line, it was hard to keep track of it since the command line was also responsible for other commands. So, we decided to build a Graphical User Interface along with the command line display. This graphical interface would only have a board displayed on it so that as we typed in commands such as play, attack, or use, we could see the board changing in real time.

The observer design pattern was an effective solution to this strategy. It also allowed us to increase code reuse of the board's state on different interfaces. So, if we wanted to add another way to display the board, it would be very easy since we would only be responsible for making that new class and attaching it to the list of observers. This increased the scalability of the project.

The observer pattern involves two components: the subject and the observers. The subject has a list of observers and it notifies them when there's a change of state. The observers then update themselves based on the change. In our program we had the board class as the concrete subject. The TextObserver class and GraphicObserver class were the concrete observers. The TextObserver class was responsible for the command line interface. It attached itself to the board and its notify method would update the display of the board. Similarly, the graphic observer would also attach itself to the board and when its notify method was called, it would use XMing to display the board on a separate window.

We used Decoupling interfaces (MVC) principles. By making the board class solely responsible for maintaining the state of the game, we maintained the Single Responsibility Principle. It did not need to know anything about how the state of the game was being displayed on different interfaces. This separation made the code more modular and easy to maintain.

Overall, the observer pattern was an effective design choice that allowed decoupling interfaces, real-time updates on the state of the board, and easy scalability for other such displays. This made the code base easier to maintain.

Resilience to Change

The utilization of inheritance in Card and concrete Card classes facilitates the resilience to change through the inherited functions and values. If a new attribute in Card is to be implemented, a simple change in the Card class would successfully add this field to all its children classes: the Minion, Enchantment, Ritual and Spell. If new card types are to be added, they would simply inherit from the parent Card class as well. Additionally, with the enforcement of our decorator pattern, changes made to individual Enchantment/Minion cards would only require adjustments in their own respective classes (e.g. changing attack values, creating a new ActivatedAbility), other classes would remain the same, promoting efficiency. For example, if we are adding a new Enchantment that grants a new Activated Ability to the Minion and overshadowing Minion's default Activated Ability, we would only need to define an ActivatedAbility object in this Enchantment object's constructor, and change within the getActivatedAbility() accessor to return the ActivatedAbility of the Enchantment object: this is all done within the Enchantment class. By doing this, when the useActivatedAbility() function is called from MinionComponent, the ActivatedAbility object returned from the accessor would be from the Enchantment rather than the Minion. All functions in the Card classes are aimed at the one goal of performing actions with the cards, such as applying abilities using apply() function and using cardAt() function for displaying the card. The high cohesion within our classes ensures localized changes, so that unintended effects outside of the card class' responsibility wouldn't occur. Similarly, if any modifications need to be made with the display of the board (e.g. changing symbols representing the border, etc.), the Observer pattern can also handle this effectively through altering the contents only within the TextObserver/GraphicObserver classes. Overall, the resilience to change in our program is also reflected by our effort in limiting coupling, as the interdependence of our classes ensures that changes made within one would not require modifications in another to maintain program's functionalities.

One limitation of our project's resilience to change might come from the adaptation of our board to the dimensions. Since the length and the width of the Board class is determined, it may be unresponsive to different screen sizes and would require many changes in the Observer classes to accommodate with different users. One solution to this is to construct a scroll bar, or to implement multiple TextObservers/GraphicObservers to display sections of the Board, allowing more flexibility in the game. Additionally, if changes are to be made with the Player class (e.g. addition of another Player object to the board), other classes' dependency on the Player objects may result in a significant amount of code change. For example, the Board class would need to update the variable tracking the active Player, and the Card classes

would need to alter their parameters to take in more Players (this could be solved by implementing a Player vector).

Answers to Questions

Question: What design pattern would be ideal for implementing enchantments? Why?

- Design pattern: decorator
- The decorator pattern allows the dynamic addition of activated ability, and/or the modifications on attack/defense stats without changing the Card's contents.
- Once enchantments are removed from the Minion, it still contains its attributes and returns to its default values.
- So with the decorator pattern, it's easy to keep track of each layer of enchantments that are added to the minion cards.

Question: How could you design activated abilities in your code to maximize code reuse?

- An ActivatedAbility class is created
- All ActivatedAbility would have a variable accounting for the costs of magic, and a function specifying the application of the activated ability. This helps maximize code reusability.

Question: Suppose we found a solution to the space limitations of the current user interface and wanted to allow minions to have any number and combination of activated and triggered abilities. What design patterns might help us achieve this while maximizing code reuse?

- Observers can be used to solve the space limitations problem.The Observer class can be set up in a way that lets the viewer see specific parts of the board, and to only modify the section of the board that changed, promoting efficiency.

Question: How could you make supporting two (or more) interfaces at once easy while requiring minimal changes to the rest of the code?

- The observer pattern can be used to support two or more interfaces at once. For example, with TextObserver and GraphicObserver as 2 of the concrete classes for observers, code reusability is made easier. This is because the only function we would need to change is the notify() function that updates how the interface looks like.

Extra Credit Features

As part of our project, we implemented a Graphical User Interface as an extra credit feature. Our goal was to make the user experience even better by providing a visual of how the board would look at all times.

One challenge that we faced before implementing a graphical interface was that it was hard to keep displaying the board on the command line. Since the command line was responsible for displaying many other things such as the hand, the minions with all their enchantments, and using play, use, and attack commands, it was not only tedious to keep displaying the board, but it also created space limitations. Since the board was so huge, we had to keep making the font size smaller to see the entire board. By implementing the graphical user interface, we were able to see the entire board in real-time as we typed in commands since the graphics were built on a separate window.

As described above, we used the observer design pattern for decoupling the interfaces, making code reuse and code maintainability easier, and making our project scalable. Along with these, we also enhanced the user experience, accessibility, engagement, and the appeal of this game.

Final Questions

1. What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

Working on the Sorcery Card Game project taught us valuable lessons about developing software in teams. We effectively divided tasks based on each team member's knowledge and capability, such as implementing the various classes, Observer, and Decorator patterns for different components of the game. Using Git allowed us to collaborate seamlessly, manage changes, and integrate various parts of the game efficiently. Working in a group,we were able to identify and fix bugs that might have gone unnoticed by an individual. This helped to catch and fix errors easily.

2. What would you have done differently if you had the chance to start over?

If we had the chance to start over, we would implement more thorough test cases from the beginning to ensure that all aspects of the game are well-tested. We would also focus on compiling and addressing errors earlier in the process to avoid larger issues later. Instead of waiting for all classes to be implemented, we would start by developing and testing key components, such as the Player and Board classes, before moving on to additional classes like Card. This approach would allow for early detection and resolution of issues. Additionally, we would incorporate testing after the creation of each class to ensure that each component functions correctly before integrating it into the main classes.

Conclusion

Overall, developing this card game sorcery gave us a deep understanding on Object Oriented programming principles and implementing various design patterns. By planning each step, starting from the UML diagrams, we were able to follow a detailed plan. The UML diagram was the most important part of the planning aspect as it provided higher level relationships between classes. This allowed us to divide the work in a way where we wouldn't have to depend on each other's work to do our own. Since we knew how different classes interacted with one another, it was much easier to work on our own parts. We also learned many software engineering skills by working on this project as a group. Building this card game was a fun way to test our skills and understanding of software engineering and object oriented programming principles.